
Module 07: Styling & Animations

Create stunning visual experiences with CSS scoping, Svelte transitions, GSAP, and ScrollTrigger.

Revolution Trading Pros - Web Dev Course

Overview

Great applications are not just functional - they are beautiful and feel alive. In this module, you will learn advanced CSS techniques in Svelte, built-in transition directives, and how to integrate GSAP (GreenSock Animation Platform) for professional-grade animations. By the end, you will build an animated landing section with scroll-triggered effects.

Advanced CSS Scoping

You already learned the basics of Svelte's scoped styles in Module 2. Now let us explore advanced patterns.

The `:global()` Modifier

The `:global()` modifier breaks out of Svelte's scoping. Use it carefully and only when necessary.

Svelte

```
<style>
  /* Scoped: only affects <p> in this component */
  p { color: #333; }

  /* Global: affects all <body> elements */
  :global(body) {
    margin: 0;
    font-family: system-ui;
  }

  /* Scoped parent, global child */
  .container :global(h2) {
    /* Affects all h2 inside .container, even from child components */
    color: #7c3aed;
  }

  /* Global block - everything inside is global */
  :global {
    .prose h1 { font-size: 2.5rem; }
    .prose p { line-height: 1.8; }
    .prose code { background: #f0f0f5; padding: 0.2rem 0.4rem; }
  }
</style>
```

Dynamic Classes and Styles

Svelte

```
<script lang="ts">
  let isActive = $state(false);
  let theme = $state<"light" | "dark">("light");
  let progress = $state(0);
</script>

<!-- class: directive -->
<div class:active={isActive}>Toggle me</div>

<!-- Multiple dynamic classes -->
<div
  class="card"
  class:active={isActive}
  class:dark={theme === "dark"}
>Content</div>

<!-- Dynamic style with CSS variables -->
<div style:--progress="{progress}%">
  <div class="bar"></div>
</div>

<style>
  .bar {
    width: var(--progress);
    height: 4px;
    background: #00d4ff;
    transition: width 0.3s ease;
  }
</style>
```

Svelte Transitions

Svelte includes built-in transition directives that animate elements as they enter and leave the DOM. These are declarative, performant, and incredibly easy to use.

Svelte

```
<script lang="ts">
  import { fade, fly, slide, scale, blur } from "svelte/transition";
  import { quintOut, elasticOut } from "svelte/easing";

  let visible = $state(true);
</script>

<button onclick={() => visible = !visible}>Toggle</button>

{#if visible}
  <!-- Basic fade -->
  <div transition:fade>Fade in/out</div>

  <!-- Fly from direction -->
  <div transition:fly={{ y: 50, duration: 400 }}>Fly up</div>

  <!-- Scale with easing -->
  <div transition:scale={{ start: 0.5, easing: elasticOut }}>Scale</div>

  <!-- Slide (accordion effect) -->
  <div transition:slide={{ duration: 300 }}>Slide</div>

  <!-- Blur -->
  <div transition:blur={{ amount: 10 }}>Blur</div>
{/if}
```

In/Out Transitions

Use in: and out: for separate enter and exit animations:

Svelte

```
{#if visible}
  <div
    in:fly={{ y: -20, duration: 400, delay: 200 }}
    out:fade={{ duration: 200 }}
  >
    Different enter and exit animations
  </div>
{/if}
```

Transition Events

Svelte

```
<div
  transition:fly={{ y: 50 }}
  onintrostart={() => console.log("enter started")}
  onintroend={() => console.log("enter complete")}
  onoutrostart={() => console.log("exit started")}
  onoutroend={() => console.log("exit complete")}
>
  Animated element
</div>
```

GSAP Integration

GSAP (GreenSock Animation Platform) is the industry-standard JavaScript animation library. It provides timeline-based animations, scroll triggers, and physics-based motion that go far beyond what CSS transitions can achieve.

Installing GSAP

Terminal

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. The text 'pnpm add gsap' is displayed in a light green monospace font.

```
pnpm add gsap
```

Basic GSAP Animations

```

<script lang="ts">
  import { gsap } from "gsap";

  let heroRef: HTMLElement;
  let titleRef: HTMLElement;
  let subtitleRef: HTMLElement;

  $effect(() => {
    const ctx = gsap.context(() => {
      // Animate title
      gsap.from(titleRef, {
        y: 60,
        opacity: 0,
        duration: 1,
        ease: "power3.out"
      });

      // Animate subtitle with delay
      gsap.from(subtitleRef, {
        y: 40,
        opacity: 0,
        duration: 0.8,
        delay: 0.3,
        ease: "power3.out"
      });
    }, heroRef); // Scope to heroRef

    // Cleanup on unmount
    return () => ctx.revert();
  });
</script>

<section bind:this={heroRef}>
  <h1 bind:this={titleRef}>Welcome</h1>
  <p bind:this={subtitleRef}>Build amazing things.</p>
</section>

```

GSAP Timelines

Timelines let you sequence multiple animations with precise control:

```
<script lang="ts">
  import { gsap } from "gsap";

  let containerRef: HTMLElement;

  $effect(() => {
    const ctx = gsap.context(() => {
      const tl = gsap.timeline({ defaults: { ease: "power3.out" } });

      tl.from(".hero-title", { y: 80, opacity: 0, duration: 1 })
        .from(".hero-subtitle", { y: 50, opacity: 0, duration: 0.8 }, "-=0.5")
        .from(".hero-cta", { scale: 0.8, opacity: 0, duration: 0.6 }, "-=0.3")
        .from(".hero-image", { x: 100, opacity: 0, duration: 1 }, "-=0.8");
    }, containerRef);

    return () => ctx.revert();
  });
</script>
```

ScrollTrigger Animations

ScrollTrigger is a GSAP plugin that triggers animations based on scroll position. It is essential for modern landing pages and storytelling experiences.

Setting Up ScrollTrigger

```

<script lang="ts">
  import { gsap } from "gsap";
  import { ScrollTrigger } from "gsap/ScrollTrigger";

  // Register the plugin
  gsap.registerPlugin(ScrollTrigger);

  let sectionRef: HTMLElement;

  $effect(() => {
    const ctx = gsap.context(() => {
      // Fade in cards as they enter viewport
      gsap.from(".feature-card", {
        y: 80,
        opacity: 0,
        duration: 0.8,
        stagger: 0.2, // 0.2s delay between each card
        ease: "power2.out",
        scrollTrigger: {
          trigger: ".features-section",
          start: "top 80%", // When top of trigger hits 80% of viewport
          end: "bottom 20%", // When bottom hits 20% of viewport
          toggleActions: "play none none reverse"
        }
      });

      // Parallax effect
      gsap.to(".parallax-bg", {
        yPercent: -30,
        ease: "none",
        scrollTrigger: {
          trigger: ".parallax-section",
          start: "top bottom",
          end: "bottom top",
          scrub: true // Ties animation to scroll position
        }
      });
    }, sectionRef);

    return () => ctx.revert();
  });
</script>

```

ScrollTrigger Options Explained

- `trigger`: The element that triggers the animation when scrolled into view.
 - `start/end`: Define the scroll range. Format: 'element-position viewport-position'.
 - `toggleActions`: Four states - onEnter, onLeave, onEnterBack, onLeaveBack. Values: play, pause, resume, restart, reset, complete, reverse, none.
 - `scrub`: When true (or a number for smoothing), ties animation progress directly to scroll position.
 - `pin`: When true, pins the trigger element in place during the animation.
 - `markers`: Set to true during development to visualize trigger points.
-

\$effect() for Animation Cleanup

Proper cleanup is critical when using GSAP in Svelte components. Without cleanup, animations can persist after the component is destroyed, causing memory leaks and visual glitches.

Svelte

```
<script lang="ts">
  import { gsap } from "gsap";
  import { ScrollTrigger } from "gsap/ScrollTrigger";

  gsap.registerPlugin(ScrollTrigger);

  let ref: HTMLElement;

  $effect(() => {
    // gsap.context() scopes all animations created inside it
    const ctx = gsap.context(() => {
      // All GSAP code here is automatically tracked
      gsap.to(".element", { x: 100 });

      ScrollTrigger.create({
        trigger: ".section",
        onEnter: () => console.log("entered")
      });

      gsap.timeline()
        .to(".a", { y: 50 })
        .to(".b", { rotation: 360 });
    }, ref);

    // ctx.revert() kills ALL animations and ScrollTriggers
    // created within the context
    return () => ctx.revert();
  });
</script>
```

The `gsap.context()` method is the recommended way to manage GSAP animations in component-based frameworks. It tracks all animations created within it and provides a single `revert()` call that cleans everything up.

Common Cleanup Mistakes

❗ Forgetting to return a cleanup function from `$effect()` - animations persist after navigation.

- Using `gsap.killTweensOf()` instead of `context` - misses `ScrollTrigger` instances.
 - Creating `ScrollTriggers` outside of `$effect()` - they run before the DOM is ready.
 - Not scoping the context to a ref element - animations may target wrong elements.
-

Module 7 Exercise

Exercise: Animated Landing Section

Put what you have learned into practice with this hands-on exercise.

- Step 1: Create a new route `/landing` with a full-page hero section.
- Step 2: Add a hero title, subtitle, and CTA button. Animate them in sequence using a GSAP timeline.
- Step 3: Create a features section below with 3-4 cards.
- Step 4: Use `ScrollTrigger` to animate the feature cards as they scroll into view with stagger.
- Step 5: Add a parallax background effect to a section using `scrub`.
- Step 6: Include `Svelte transition:fly` on toggle-able elements (e.g., a modal or dropdown).
- Step 7: Use CSS custom properties for theming (colors, spacing).
- Step 8: Ensure all GSAP animations are properly cleaned up with `gsap.context()` and `$effect()`.
- Step 9: Test the animations at different scroll speeds and viewport sizes.
- Step 10: Commit your changes to Git.

Summary

- `:global()` breaks out of Svelte's CSS scoping when needed.
- Svelte transitions (`fade`, `fly`, `slide`, `scale`) animate DOM enter/exit.
- GSAP provides professional-grade timeline and physics-based animations.
- `ScrollTrigger` ties animations to scroll position for immersive experiences.
- `$effect()` with `gsap.context()` ensures proper animation lifecycle management.
- Always return a cleanup function from `$effect()` to prevent memory leaks.